

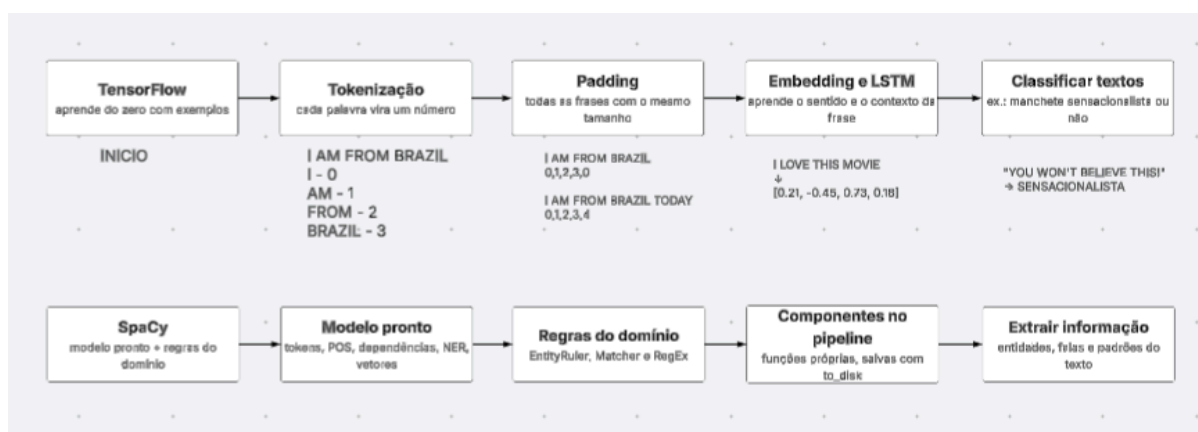
Relatório 25 - Prática: Processamento de Linguagem Natural (NLP) (III)

Igor Carvalho Marchi

1. Introdução

Este card trata de Processamento de Linguagem Natural (NLP), a área que busca fazer o computador ler, entender e extrair informações de textos escritos por pessoas. O conteúdo foi dividido em dois cursos: a playlist NLP Zero to Hero, do TensorFlow, que mostra como transformar texto em números e treinar redes neurais com ele, e o curso Natural Language Processing with spaCy & Python, que apresenta a biblioteca spaCy e as abordagens baseadas em regras.

2. Desenvolvimento

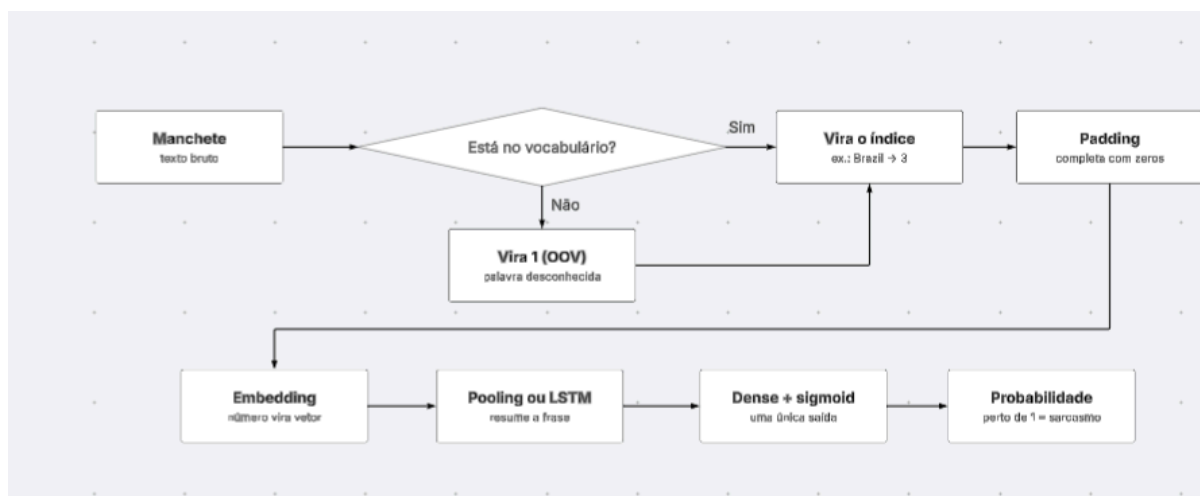


A imagem acima tem dois caminhos para o computador entender texto. No TensorFlow (linha de cima) o modelo começa do zero: a frase "I am from Brazil" vira números, as frases são igualadas pelo padding, viram vetores no embedding e só então o modelo aprende a classificar, por exemplo, se "You won't believe this!" é uma manchete sensacionalista. No spaCy (linha de baixo) já existe um modelo pronto, e o trabalho é completá-lo com regras e componentes para extrair informação. Os dois se completam: um entende a estrutura do texto e o outro aprende a tomar decisões sobre ele.

2.1 NLP Zero to Hero (TensorFlow)

Tokenização. O computador só trabalha com números, então o primeiro passo é converter o texto. Codificar letras (como na tabela ASCII) não funciona bem, porque "LISTEN" e "SILENT" teriam os mesmos números em ordem diferente. Por isso cada palavra recebe um número. A classe Tokenizer faz isso: `num_words` limita o tamanho do vocabulário, `fit_on_texts` lê as frases e `word_index` devolve o dicionário palavra → número. Ele já converte tudo para minúsculas e ignora a pontuação, então "dog" e "dog!" recebem o mesmo número, e as palavras mais frequentes ficam com os menores índices.

Sequências, OOV e padding. Com `texts_to_sequences` cada frase vira uma lista de índices. Palavras que não estavam no vocabulário seriam simplesmente descartadas, o que muda o sentido da frase. Para evitar isso usa-se o `oov_token` ("Out Of Vocabulary"), que coloca um índice reservado no lugar da palavra desconhecida. Como a rede precisa de entradas do mesmo tamanho, o `pad_sequences` completa as sequências com zeros (antes ou depois, pelo parâmetro `padding`) e corta as maiores com `maxlen` e `truncating`.



Fluxograma do texto até a previsão da rede

Em nenhum momento a rede vê palavras, cada palavra conhecida vira o seu índice no vocabulário, toda palavra desconhecida vira o mesmo número 1 (OOV) e o padding deixa todas as frases do mesmo tamanho. Por isso o vocabulário montado no treino limita o que o modelo consegue entender depois: uma gíria nova vira apenas "1".

Classificação de sarcasmo. Foi usada uma base de manchetes com os campos `headline`, `is_sarcastic` e `article_link`, separada em 20.000 frases de treino e o restante de teste. O tokenizer é ajustado só com o treino, para o modelo não "ver" o vocabulário do teste. O modelo tem uma camada `Embedding`, que transforma cada índice em um vetor que é aprendido no treino, e palavras com sentido parecido acabam com vetores próximos. Depois vêm um `GlobalAveragePooling1D`, que tira a média dos vetores da frase, uma camada `Dense` com `ReLU` e uma saída `sigmoid` com perda `binary_crossentropy`. Os gráficos de treino mostraram a acurácia de treino subindo enquanto a perda de validação aumentava, sinal de `overfitting`. Os vetores aprendidos foram exportados (`vecs.tsv` e `meta.tsv`) para visualização no `Embedding Projector`.

LSTM. Em uma rede recorrente simples, a informação do início da frase se perde conforme a frase fica longa. A LSTM mantém um estado de célula, uma espécie de memória que decide o que guardar e o que descartar a cada palavra. Com `Bidirectional` a frase é lida nos dois sentidos, e com `return_sequences=True` é possível empilhar uma LSTM sobre a outra.

2.2 NLP com spaCy

2.2.1 Doc, Token e Span

No spaCy, o texto processado fica guardado em um objeto chamado **Doc**. Ele é formado por **Tokens**, cada um uma palavra ou pontuação com função própria, e por **Spans**, trechos de um ou mais tokens, como "Martin Luther King". O tokenizador do spaCy é mais inteligente que o `split()` do Python: separa pontuação, parênteses e contrações, e divide o texto em sentenças (`doc.sents`) sem se confundir com abreviações como "Mr.".

2.2.2 Anotações de cada token

Cada token já vem com informações linguísticas calculadas pelo modelo. No exemplo da aula, a palavra "States" em "The United States of America" recebe:

Atributo	O que mostra	Exemplo com "States"
<code>pos_</code>	classe gramatical	PROPN (nome próprio)
<code>dep_</code>	função na frase	nsubj (sujeito)
<code>ent_type_</code>	tipo de entidade	GPE (entidade geopolítica)
<code>lemma_</code>	forma base da palavra	States

O displacy mostra essas informações de forma visual: desenha a árvore de dependências e destaca as entidades em cores.

2.2.3 Entidades e vetores de palavras

O reconhecimento de entidades (NER) encontra nomes no texto e os classifica com rótulos como PERSON (pessoa), ORG (organização), GPE (país ou cidade), DATE (data) e CARDINAL (número). Já os vetores de palavras representam cada palavra como uma lista de números, de modo que palavras com sentido parecido fiquem próximas. O modelo pequeno (en_core_web_sm) não tem vetores, então foi usado o médio (en_core_web_md) para buscar as palavras mais parecidas com "country" e comparar frases com similarity.

2.2.4 Pipeline

O modelo do spaCy funciona como uma linha de produção: o texto passa por vários componentes (pipes) em ordem, e cada um acrescenta uma informação ao Doc.

1. tok2vec: transforma os tokens em vetores de contexto
2. tagger: marca a classe gramatical
3. parser: monta a árvore de dependências
4. attribute_ruler e lemmatizer: ajustam atributos e encontram a forma base
5. ner: reconhece as entidades

Também é possível começar com um pipeline vazio (spacy.blank("en")) e colocar só o necessário, como o sentencizer, o que deixa o processamento bem mais rápido. O analyze_pipes mostra o que cada pipe faz e de quais outros depende.

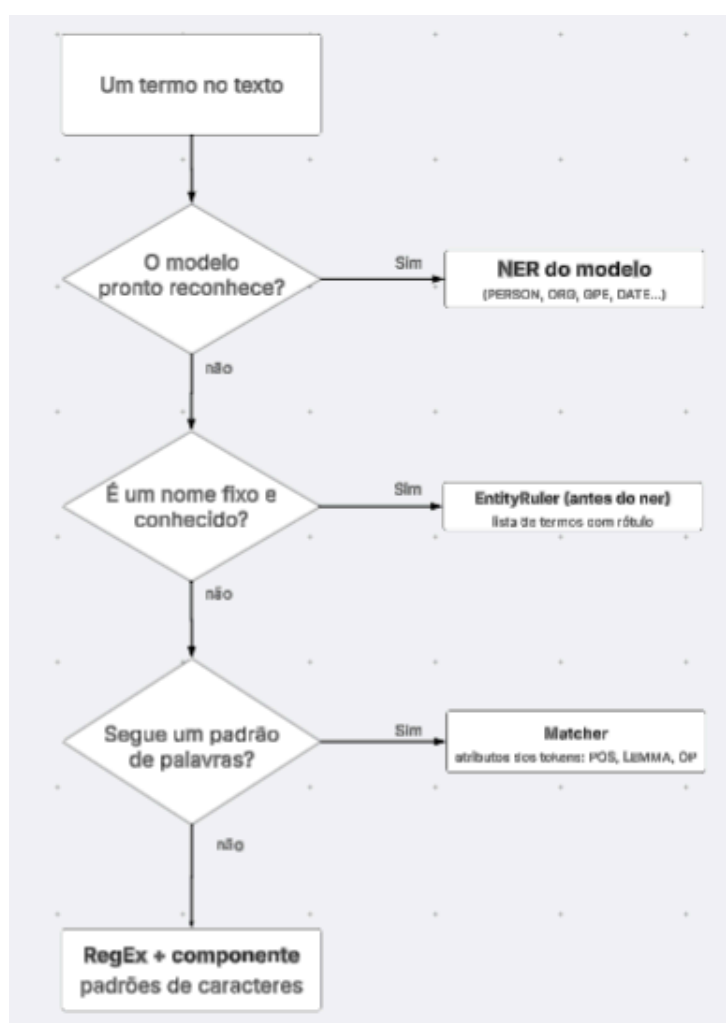
2.2.5 Abordagens por regras

Quando o modelo pronto não conhece o vocabulário de uma área, entram as regras. Cada ferramenta resolve um tipo de problema:

Ferramenta	Como funciona	Exemplo da aula
EntityRuler	lista de termos com rótulo; antes do ner, tem prioridade sobre o modelo	"West Chestertenfieldville" marcado como GPE
Matcher	procura sequências de tokens por atributos (POS, LEMMA, ORTH) com os operadores +, * e ?; o greedy="LONGEST" fica com a maior	falas e falantes em <i>Alice no País das Maravilhas</i>

Componente customizado	função Python registrada com <code>@Language.component</code> e adicionada ao pipeline	componente que remove as entidades GPE
RegEx	padrões de caracteres, aplicados com <code>re.finditer</code> e convertidos em Span com <code>doc.char_span</code>	número de telefone e nomes como "Paul Newman"

O RegEx é rápido e flexível, mas tem sintaxe difícil. Como o EntityRuler não aplica RegEx entre vários tokens, o resultado é inserido nas entidades com `filter_spans`, que resolve sobreposições ficando com o trecho mais longo.



Fluxograma de decisão: qual ferramenta usar

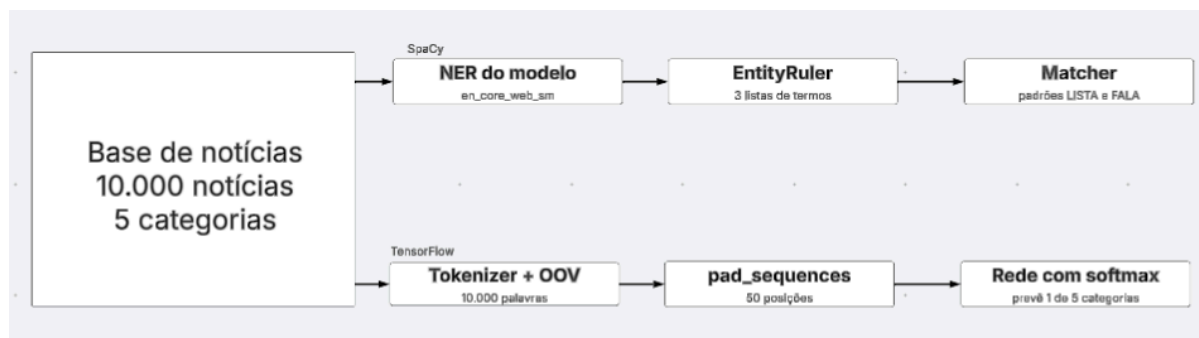
Os nomes fixos vão para o EntityRuler, colocado antes do ner para ter prioridade. Sequências que dependem da gramática vão para o Matcher. Padrões de caracteres, como um número de telefone, precisam de RegEx dentro de um componente customizado.

Por fim, essas técnicas foram juntadas em uma aplicação financeira: listas de ações, empresas, índices e bolsas de valores (arquivos .tsv) viraram padrões de um EntityRuler com os rótulos STOCK, COMPANY, INDEX e STOCK_EXCHANGE, aplicado a uma notícia da Reuters.

3. Prática: análise e classificação de notícias

Na prática foi usada a base News Category Dataset (Kaggle), com notícias do HuffPost. Foram escolhidas cinco categorias (POLITICS, SPORTS, TECH, TRAVEL e FOOD & DRINK), com 2.000 notícias de cada, usando a manchete junto com a descrição curta como texto.

No spaCy, o `en_core_web_sm` foi usado para ver os atributos dos tokens e as entidades das notícias. Depois foi criado um EntityRuler com listas de ligas esportivas, empresas de tecnologia e partidos políticos, e um Matcher com dois padrões de manchete: as que começam com número ("10 Best Beaches") e as de nome próprio seguido de verbo de fala ("Trump Says"). No TensorFlow, as notícias passaram pelo Tokenizer e pelo padding e treinaram o mesmo modelo da aula de sarcasmo, adaptado com softmax para classificar as cinco categorias.



Fluxo da prática

A mesma base passa por dois caminhos: no spaCy são extraídas entidades e padrões das manchetes, e no TensorFlow a rede aprende a prever a categoria de cada notícia.

4. Conclusão

Os dois cursos se complementam. O do TensorFlow mostra o primeiro passo de qualquer sistema de PLN, transformar palavras em números com tokenização, sequências e padding, e como uma rede com embeddings e LSTM aprende a partir disso. O do spaCy mostra um pipeline completo que já entrega lemas, classes gramaticais, dependências, entidades e vetores, e como complementá-lo com regras.

Ficou claro que modelos prontos funcionam bem para casos gerais, mas problemas de domínio específico pedem regras próprias, como EntityRuler, Matcher e RegEx, e que a ordem dos pipes define qual resultado prevalece. Na prática com as notícias, esses conceitos foram aplicados em uma base real, e a mesma estrutura do classificador de sarcasmo foi adaptada para um problema com cinco categorias.

5. Referências

- [Natural Language Processing with spaCy & Python - Course for Beginners](#) – freeCodeCamp
- [Natural Language Processing - Tokenization \(NLP Zero to Hero - Part 1\)](#) – TensorFlow
- <https://www.kaggle.com/datasets/rmisra/news-category-dataset> – Rishabh Misra, Kaggle(usado na pratica).
- [https://lucid.app/lucidspark/18066335-de30-4eed-9880-54abe681d706/edit?viewport_loc=226000%2C-13700%2C131200%2C61000%2C3B0CR\\$ot8otl&invitationId=inv_21f49f9f-9f89-43de-80e8-b16ebb14e097](https://lucid.app/lucidspark/18066335-de30-4eed-9880-54abe681d706/edit?viewport_loc=226000%2C-13700%2C131200%2C61000%2C3B0CR$ot8otl&invitationId=inv_21f49f9f-9f89-43de-80e8-b16ebb14e097) - Link dos Insights visuais